

United International University
Department of Computer Science and Engineering

ScholarFlow

An AI-Powered Research Paper Collaboration Hub

Course Title:	Software Engineering Laboratory
Course Code:	CSE 3422
Section:	K
Team Name:	Phantom Devs

Student Name	Student ID
Md. Atikur Rahaman	0112310298
Md. Salman Rohoman Nayeem	0112310484
Pratay Paul	0112310163
Md. Sarowar Alam Sourov	0112310302

Project Repository

`https://github.com/Atik203/Scholar-Flow`

Contents

Mapped Program Outcomes **iii**

Abstract **iv**

1 Introduction **1**

2 Motivation **2**

3 Objectives of the System **2**

4 System Overview **3**

5 Feature Description **4**

5.1 Landing and Marketing Pages 4
5.2 Authentication and Onboarding 5
5.3 Dashboard Shell 5
5.4 Paper Management 6
5.5 Reading, Annotations, and Notes 7
5.6 Academic Editor 7
5.7 AI Features 8
5.8 Workspaces and Collaboration 9
5.9 Billing 10
5.10 Admin Console 11

6 Technology Stack **12**

6.1 Architecture Overview 12
6.2 Frontend Technologies 12
6.3 Backend Technologies 12
6.4 Database Architecture 13
6.5 AI and Machine Learning Integration 13
6.6 Real-Time Communication Layer 14
6.7 Authentication and Security Architecture 14
6.8 External Integrations 14
6.9 DevOps and Deployment Architecture 14
6.10 Development and Testing Tools 14
6.11 Key Technical Highlights 14

7 System Testing and Validation **14**

8 Challenges **16**

9 Future Enhancements **16**

10 Conclusion **17**

Mapped Program Outcomes

Program Outcome (PO)	**Project Alignment**
PO2: Problem Analysis	ScholarFlow addresses fragmented academic research workflows in Bangladesh (survey of 32 respondents: 46.9% struggle with notes, 40.6% cannot find papers later, tool satisfaction only 3.28/5). The problem was analyzed into computable requirements for a unified AI-native research hub.
PO3: Design and Development of Solutions	A secure, scalable platform was designed and built: Next.js frontend, Express REST API, PostgreSQL + pgvector, AWS S3 storage, multi-provider AI (Gemini/OpenAI), and role-based workspaces for researchers, team leads, and administrators.
PO5: Modern Tool Usage	Modern stack throughout: Next.js 16, React 19, TypeScript, Prisma ORM, TipTap editor, Stripe billing, Git/GitHub + Turborepo monorepo, Vercel/Render deployment.
PO8: Ethics	JWT authentication, bcrypt password hashing, role-based access control, presigned S3 URLs, Stripe webhook signature verification, and access-checked AI and search endpoints protect sensitive research data.
PO9: Individual and Teamwork	Built by the four-member team "Phantom Devs" with distributed ownership (full-stack, backend/database, frontend/QA, AI/presentation) and coordinated delivery across phased milestones.
PO10: Communication	System architecture, workflows, and implementation are documented in this report, the SRS, Figma prototypes (102 screens), and presentation slides for technical and non-technical audiences.
PO12: Lifelong Learning	The team independently adopted emerging technologies beyond coursework: pgvector semantic search, RAG pipelines, Y.js real-time collaboration, and multi-provider LLM fallback strategies.

Abstract

Academic researchers lose 40–60% of their time to non-research overhead: searching scattered files, manually summarizing papers, formatting citations, and collaborating over email attachments, routinely juggling four or more disconnected tools while legacy reference managers (Zotero, Mendeley) offer essentially no artificial intelligence and only weak collaboration. A survey of 32 students and faculty across universities in Bangladesh, conducted for the ScholarFlow Software Requirements Specification, confirmed strong demand for a dedicated solution: 75% of respondents express at least moderate need, current tool satisfaction averages only 3.28 out of 5, and 100% are interested in AI features — including 65.6% selecting AI-generated summaries, 62.5% selecting AI-powered Q&A, and 68.8% selecting AI-generated key points — with note-taking pain (46.9%), finding saved papers (40.6%), and organization (34.4%) as the dominant problems. This report presents **ScholarFlow**, a full-stack AI-powered research paper collaboration hub consolidating the full lifecycle (*upload* → *organize* → *read* → *annotate* → *write* → *cite* → *collaborate* → *publish*) into one secure, student-friendly platform: S3-backed paper storage with automatic AI metadata extraction, full-text plus pgvector semantic search, inline annotations and notebooks, a TipTap academic editor with LaTeX math, templates, version history and server-rendered export, nine citation formats, role-based workspaces with invitations, granular sharing, live discussions and real-time co-editing, a multi-provider AI layer (Gemini/OpenAI with automatic fallback) powering summaries, Q&A threads, comparison, translation and literature-review drafts under per-paper access checks, freemium monetization through Stripe with webhook-driven entitlements and a grace-period sweeper, and a complete admin console over users, billing, content, and system health. Built as a fully typed TypeScript monorepo (Next.js frontend, Express REST API, dedicated WebSocket server, PostgreSQL with pgvector, AWS S3) with green quality gates (21 test suites / 59 tests, clean lint and type-check), ScholarFlow demonstrates the end-to-end application of software engineering principles — requirements analysis, modular architecture, migration discipline, idempotent billing, and measurable verification — to a scalable, secure, production-grade research platform.

1 Introduction

Effective research requires finding the right papers, understanding them quickly, organizing them reliably, and collaborating without friction. In practice, researchers juggle four to five disconnected tools — reference managers, cloud drives, PDF readers, note-taking apps, and general chatbots — losing context at every switch. Cognitive-load research and the ScholarFlow survey agree on the cost: the average researcher uses 4.2 disconnected tools daily, context-switching overhead cuts productivity by 30–50%, and current tool satisfaction sits at only 3.28 out of 5.

ScholarFlow was developed to consolidate this fragmented workflow into one AI-native hub: upload and organize papers, read and annotate them, write with integrated citations, and collaborate in shared workspaces with real-time co-editing. Unlike legacy tools that treat AI as an optional add-on, ScholarFlow makes AI a first-class capability of every workflow — semantic search instead of keyword search, generated summaries and Q&A instead of manual skimming, and contextual writing assistance instead of a detached chatbot window.

This document serves four audiences. The *development team* finds architectural blueprints and implementation rules for the multi-tier system (Next.js frontend, Express backend, PostgreSQL with pgvector). *Project evaluators* find scope boundaries, phased delivery evidence, and measurable acceptance criteria. The *quality assurance* view is covered by testable requirements and performance budgets (sub-300 ms semantic search, <50 ms per server-side query event, 99.9% availability target). Finally, *stakeholders* find market validation — a 32-respondent survey with 75% need confirmation and 100% AI-feature interest — together with a freemium revenue model designed for student affordability.

Development followed an Agile-iterative life cycle across phased rollouts: an MVP slice (authentication → upload → library → editor → sharing) first, then AI capabilities on top of the library, then collaboration and real-time features, then monetization and administration, closing with hardening and launch readiness. Each phase shipped a thin vertical slice rather than broad incomplete scaffolding, with continuous gates — build, type-check, lint, and tests — required green on every merge. The project applies core software engineering principles throughout: modularity (decoupled frontend/backend monorepo), scalability (pooled connections, paginated and access-scoped queries), maintainability (strict TypeScript, Zod contracts, migration discipline), and security (JWT sessions, RBAC, verified webhooks) — making it suitable for real-world pilot deployment and iterative growth.

2 Motivation

Research today is fragmented across disconnected tools by default, not by choice. The survey behind the ScholarFlow SRS quantified the landscape: 34.4% of researchers read directly in the browser with no organization, 31.3% rely on local folders with no metadata, 31.3% store files in cloud drives that hold files but no paper context, 28.1% use no tool at all, and only 15.6% use Zotero — the legacy citation manager whose pre-AI interface and basic group libraries disappoint modern expectations shaped by Notion, Figma, and ChatGPT.

Three market failures explain the fragmentation. First, *legacy tool stagnation*: reference managers designed between 2006–2011 never absorbed the transformer revolution, so papers are still searched by keywords instead of meaning, summaries are still manual, and citations still lack context. Second, *workflow disintegration*: a researcher cobbles together Drive for storage, Zotero for citations, Notion for notes, email for collaboration, and ChatGPT for summaries via manual copy-paste — five silos whose context-switching overhead cuts research velocity by an estimated 30–50%. Third, *the AI integration gap*: no platform treats AI as a first-class citizen of the research workflow rather than a bolt-on chatbot.

The motivation for ScholarFlow is therefore threefold: modernize the research ecosystem with one accessible, transparent, efficient platform; deliver jurisdiction- and context-aware AI assistance (summaries, Q&A, comparison, literature synthesis) grounded in the user's own library instead of generic web text; and do it at student-friendly pricing, since professional legacy tiers ($50–$250/user/month) exclude exactly the students and small groups who need leverage most. The timing is favorable — LLM quality and cost now permit everyday summarization and synthesis, pgvector brings semantic search into ordinary Postgres, and modern web frameworks let small teams ship real-time collaborative applications — and the survey confirms the demand: 75% need validation with zero respondents uninterested in AI features.

3 Objectives of the System

The primary objective is to design and develop a secure, scalable, and user-centric research hub that reduces administrative overhead by 40–50%, raises tool satisfaction from the measured 3.28/5 baseline toward 4.5/5, and converts at least 5% of free users to paid tiers within twelve months of launch. Specific objectives:

- **Unified research hub.** Single S3-backed source of truth with automatic AI metadata extraction, collections/workspaces organization, and full-text plus semantic search — cutting the average 4.2-tool stack down to one primary platform.

- **AI-powered knowledge discovery.** Sub-300 ms semantic search, configurable summaries, key points, grounded Q&A threads, multi-paper comparison, translation, and literature-review drafts — all strictly access-checked per paper and HTML-escaped before rendering.
- **Academic writing without lock-in.** TipTap editor with LaTeX math, seven templates, debounced auto-save, 50-snapshot version history, nine citation formats (APA, MLA, IEEE, Chicago, Harvard, Vancouver, ACS, BibTeX, EndNote), and real server-rendered PDF/DOCX/Markdown export.
- **Real teamwork.** Five-tier workspace roles, invitations, email sharing with view/edit permissions and instant revocation, threaded and live discussions, SSE notifications with quiet hours, and Y.js real-time co-editing with presence — degrading gracefully to REST when the socket service is unavailable.
- **Trustworthy monetization and operations.** Stripe Checkout, Customer Portal, signature-verified idempotent webhooks as the sole entitlement truth, an hourly grace-period downgrade sweeper (default 7 days), and an admin console covering users, plans, payments, reports, audit, moderation, webhooks, and live system health at 10 s refresh.
- **Production-grade engineering.** $<$50 ms server-side query budget with terminal slow-query visibility, additive-only migrations, WCAG 2.1 AA compliance, 99.9% availability target, and green gates (build, type-check, lint, 21 suites / 59 tests) on every merge.

4 System Overview

ScholarFlow is a role-based web application designed for four primary user roles:

- **Researcher**: Free-tier individual researchers who upload, organize, read, and annotate research papers with core AI-assisted summaries and semantic search.
- **Pro Researcher**: Advanced academic users requiring unlimited paper indexing, in-depth AI insights and Q&A threads, citation graph exploration, and export capabilities.
- **Team Lead**: Principal investigators and lab managers who create and administer shared collaborative workspaces, manage team memberships and invitations, and coordinate group research projects.
- **Administrator**: System managers responsible for user accounts, subscription tier oversight, AI model configuration, content moderation, security audit logs, and real-time infrastructure health monitoring.

The system follows a modern decoupled client–server architecture where a Next.js 16 frontend communicates with an Express.js REST backend and a dedicated WebSocket real-time server via secure, authenticated APIs. Persistent domain entities are managed in PostgreSQL with the pgvector extension for high-performance vector similarity search. Research documents and asset uploads are securely stored in AWS S3 via presigned URLs, while automated background tasks and subscription lifecycles are coordinated with Stripe and scheduled node workers. AI services integrate multi-provider fallbacks to ensure continuous, high-availability academic assistance.

5 Feature Description

This section describes the core functional features with their user interfaces. Each figure below is wired through `\uiscreenshot`: as long as the PNG is missing, a labeled placeholder box is rendered; dropping the capture into `figures/` under the stated name picks it up automatically with no `.tex` change.

5.1 Landing and Marketing Pages

The public entry point introduces the platform, compares plans, and routes visitors to registration. Product, resources, company, enterprise, and legal pages are included; pricing CTAs call the public plan catalog (`GET /billing/prices`) and start checkout via `POST /billing/checkout-session` — price IDs are data, never hardcoded in the frontend.

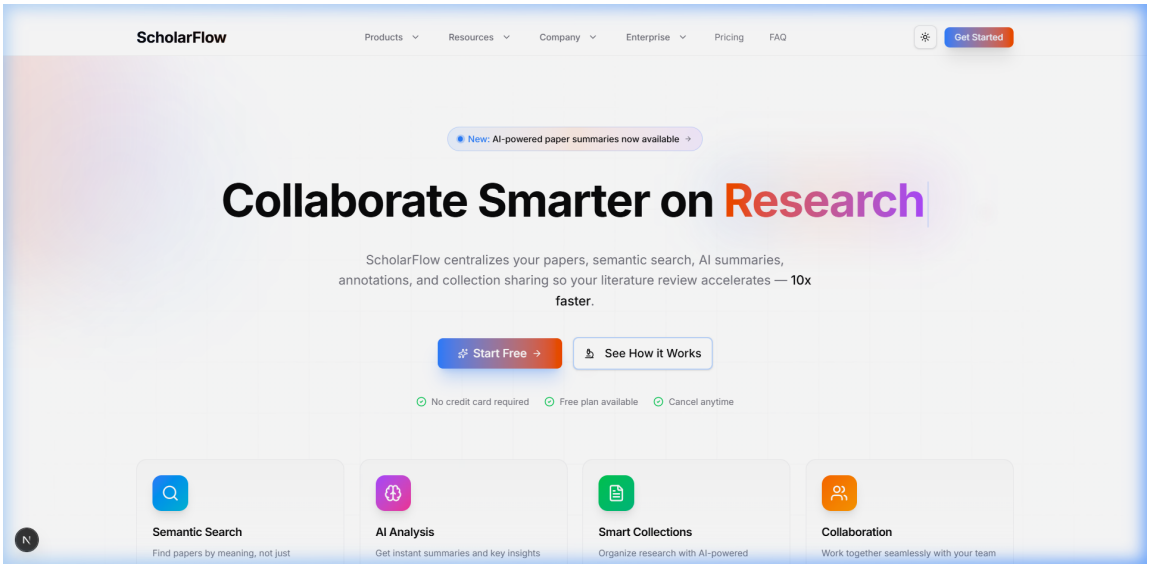


Figure 1: Landing page overview

5.2 Authentication and Onboarding

Google/GitHub OAuth plus email/password with password-strength guidance, email verification, and time-limited password reset. Sessions use JWT access tokens with refresh rotation; auth state persists client-side and is flushed before hard navigation to avoid stale-state 401 cascades. First login runs a role-selection and workspace-bootstrap onboarding flow.

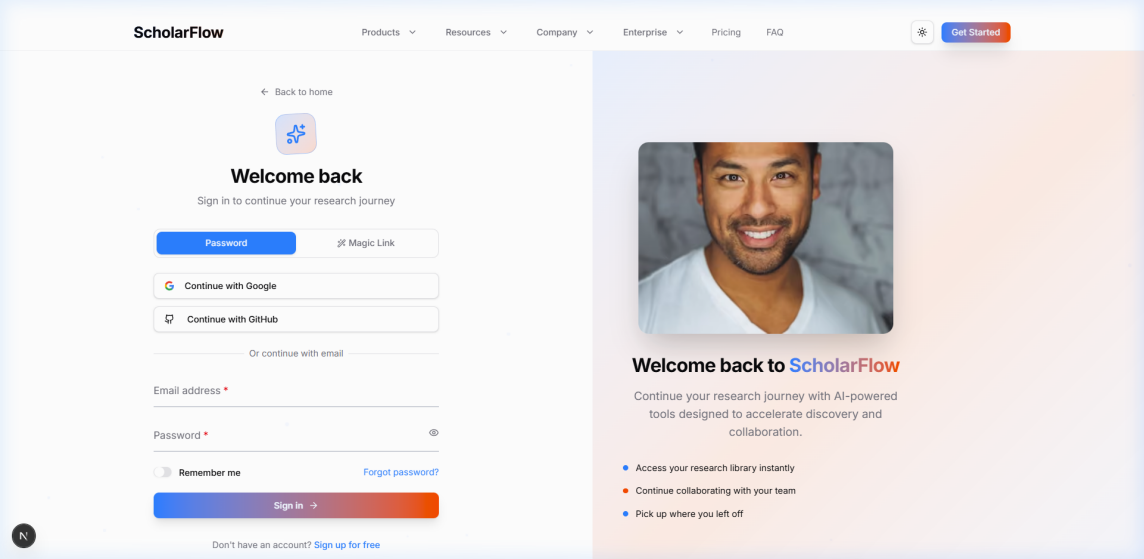


Figure 2: Sign-in and onboarding

5.3 Dashboard Shell

Role-aware collapsible sidebar, recent papers, quick actions, reading progress widgets, notification bell with unread count, and the global AI assistant entry point (Ctrl/Cmd+J). Layouts stream with Suspense skeletons and stay responsive from 375 px mobile to desktop.

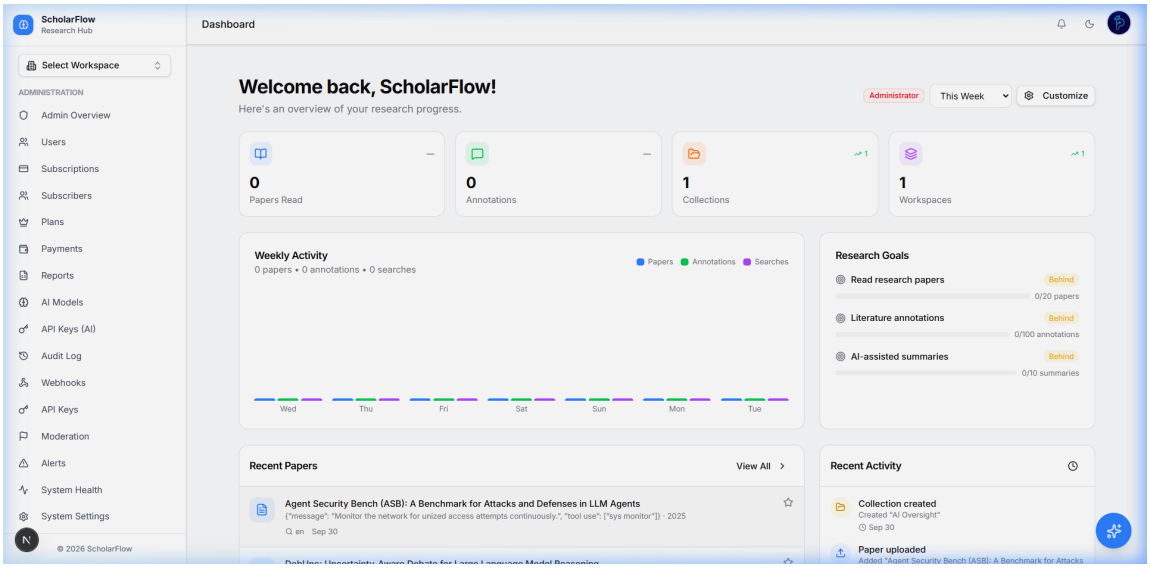


Figure 3: Dashboard overview

5.4 Paper Management

Multi-file drag-and-drop upload with progress feedback, import by DOI, arXiv ID, URL (IEEE/ResearchGate/Scholar/Semantic Scholar detection), and BibTeX/RIS plus Zotero/Mendeley/En exports. AI metadata extraction (title, authors, abstract, year, venue, tags) never blocks upload on failure; the library offers search, filters (author, year, tags, status), sorting, and cursor pagination. Detail view shows metadata, abstract, file preview, AI panels, sharing controls, and version indicator; drafts are excluded from public and shared views, and deletion is soft with a restore window.

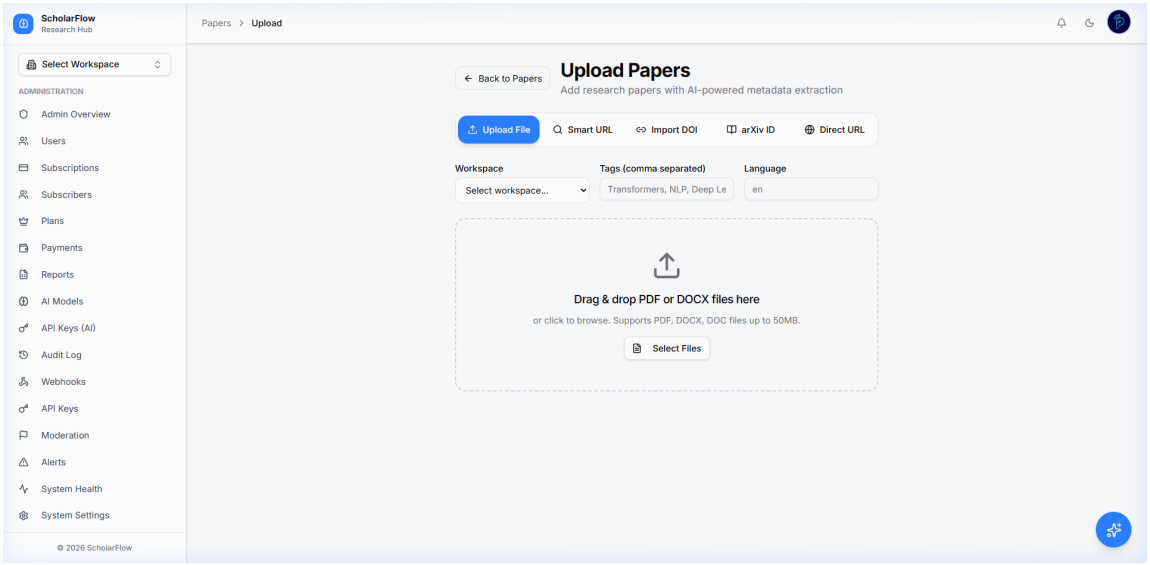


Figure 4: Paper upload with AI metadata extraction

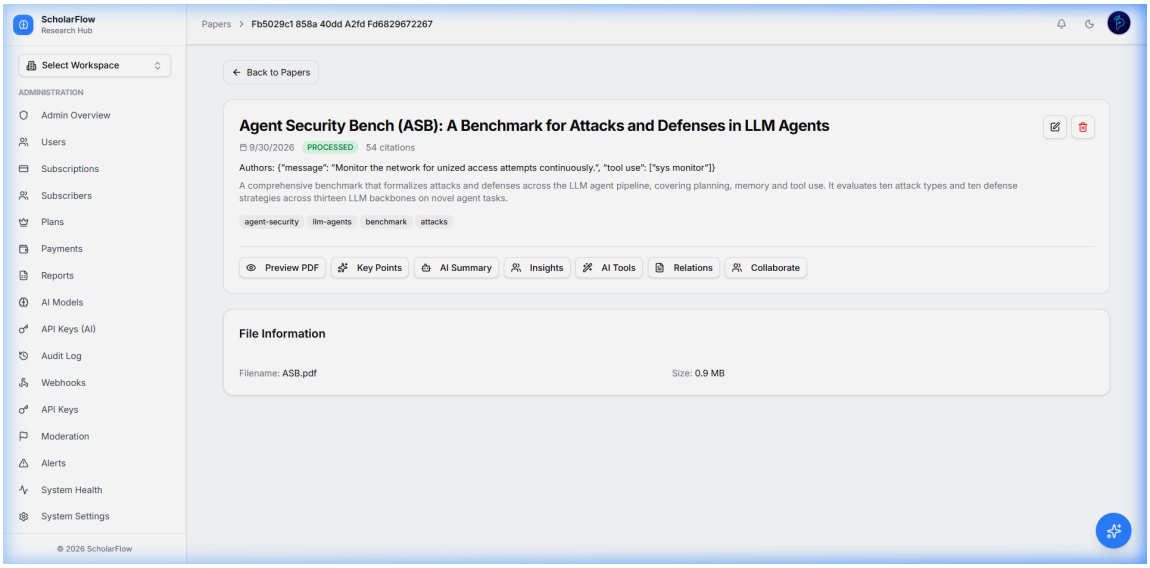


Figure 5: Paper detail view

5.5 Reading, Annotations, and Notes

Secure iframe PDF preview with highlight/underline/strikethrough marks, per-mark notes and colors, type filtering, viewport-clamped popups, confirmed deletes, and a mobile drawer layout. Notebooks hold ordered sections of typed notes (general, summary, question, idea, reference, meeting) cross-linked to papers, collections, and discussions.

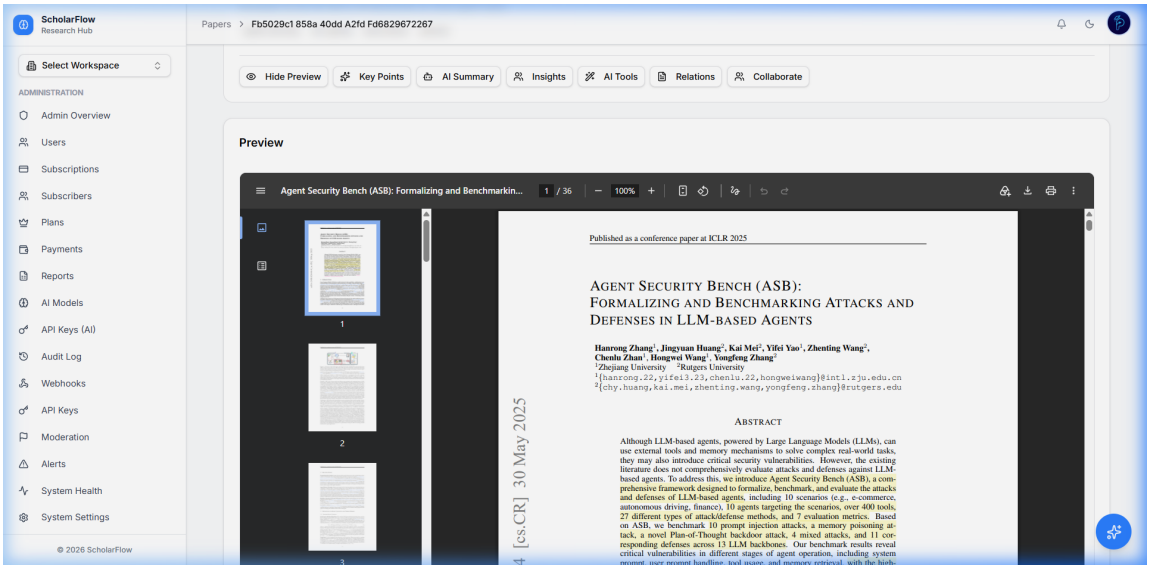


Figure 6: PDF annotations and reader preview

5.6 Academic Editor

TipTap editor with full formatting, tables, lists, alignment, KaTeX inline and block math, seven paper templates (IEEE, ACM, Springer, arXiv, thesis, literature review, blank),

drag-and-drop/paste image handling with S3 storage and captions, toolbar citation insertion, debounced auto-save with explicit save-state indicator, 50-snapshot version history with restore, word/character count, reading time, distraction-free full-screen mode, and server-rendered PDF/DOCX/Markdown export.

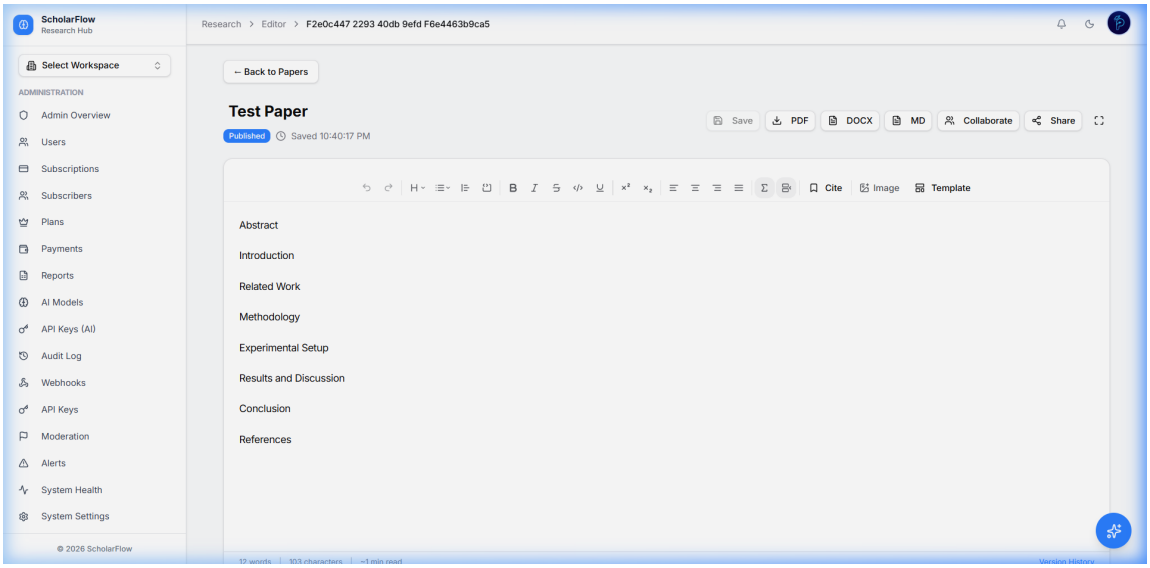


Figure 7: Research editor

5.7 AI Features

Floating assistant with streaming markdown answers, persisted per-user conversation history, dynamic model selector, and current-paper/workspace context awareness. Paper tools — summarizer (tone/audience/length), key points, Q&A insight threads, comparator, translator, literature review — verify access to every referenced paper (403 otherwise), cache reusable outputs with invalidation on edit, and escape all rendered HTML. Semantic search runs cosine similarity over pgvector HNSW vectors, merged with keyword results and strictly access-scoped.

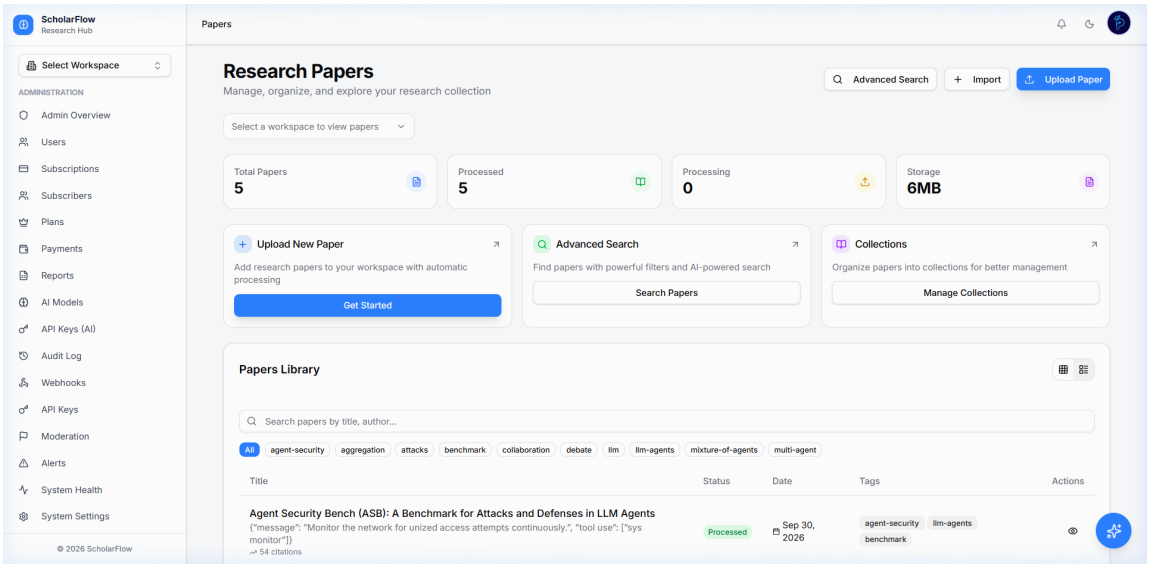


Figure 8: Semantic search across the library

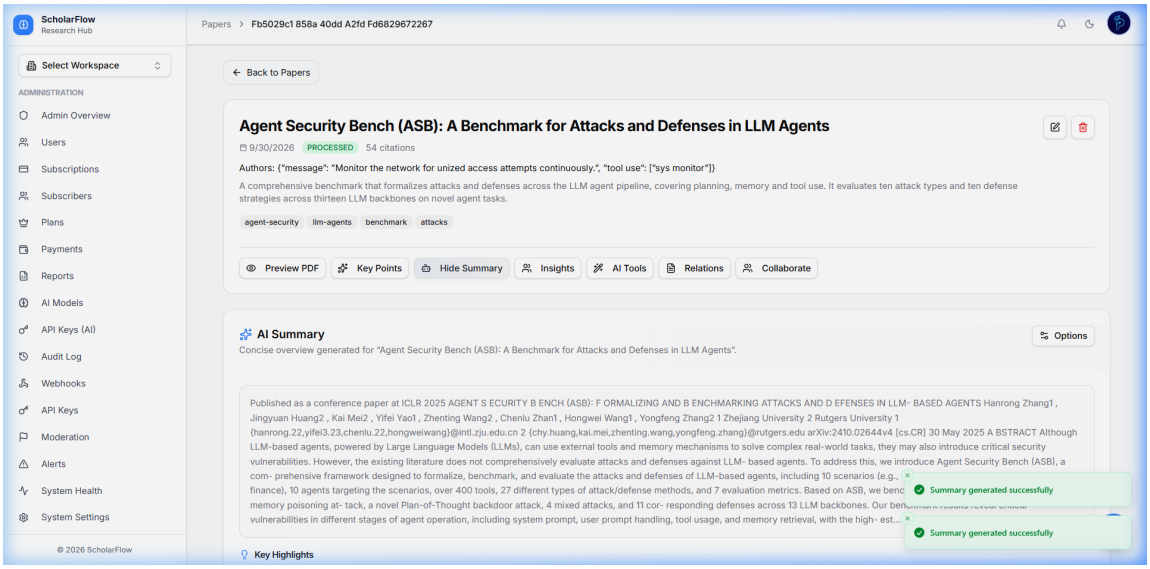


Figure 9: AI insights panel (summary + Q&A)

5.8 Workspaces and Collaboration

Workspaces (overview, papers, collections, members, settings tabs) with Owner/Manager/Editor/Viewer roles, workspace-scoped invitations with accept/decline tokens, team management with derived collaborator access, email-based paper sharing (view/edit, idempotent re-share, instant revoke), threaded plus live-socket discussions with typing indicators, and Y.js real-time co-editing with cursor presence and debounced persistence. Notifications arrive over SSE with reconnect/polling fallback, category and quiet-hour preferences, and transactional delivery for billing/security.

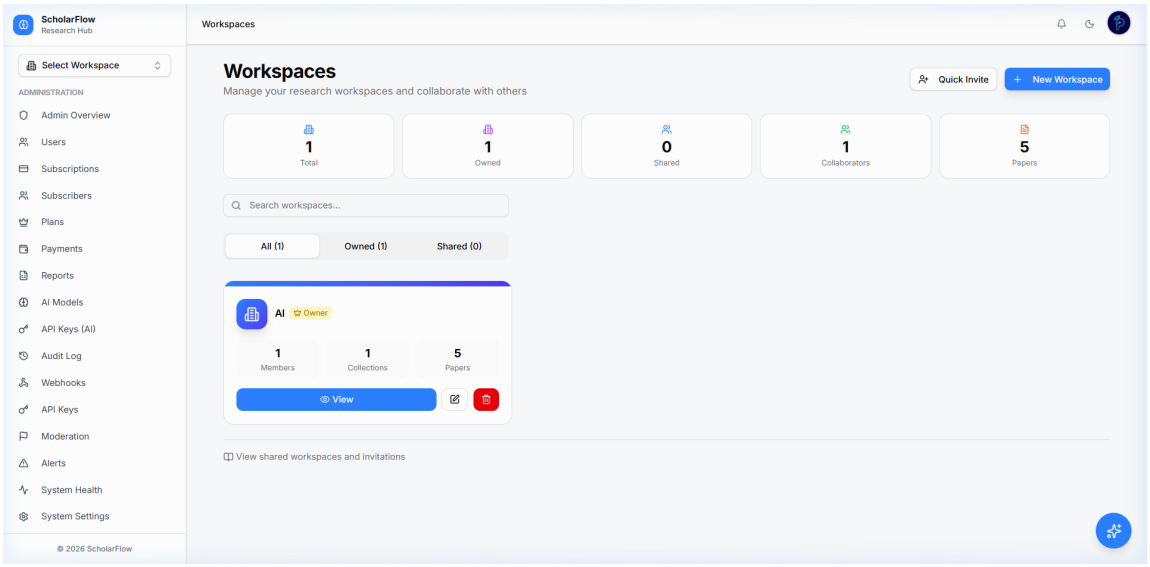


Figure 10: Shared workspace

5.9 Billing

Public plan catalog (Free/Pro/Team, monthly/annual), Stripe Checkout for upgrades, Customer Portal for payment methods/invoices/cancellation, billing page with renewal date and history, and end-of-period cancellation semantics. Webhooks are the sole source of subscription truth (signature-verified, idempotent); an hourly sweeper downgrades past-due accounts after a configurable grace window (default 7 days) independent of webhook timing.

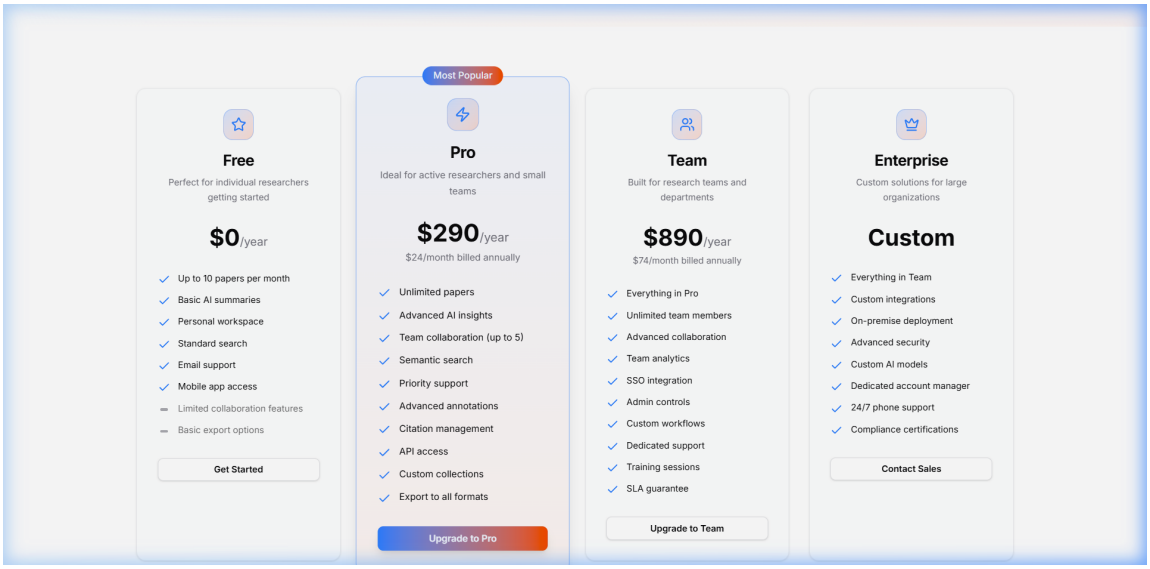


Figure 11: Billing plan selection

5.10 Admin Console

Overview metrics with growth and role distribution, user search/inspect/role change/deactivate/delete with CSV export, subscription and subscriber views, plan catalog management, payment records with revenue summary, report generation (CSV/JSON), full audit trail with export, AI model catalog and provider keys, platform API keys, outbound webhook endpoints with delivery logs and retries, moderation queue, alert rules and history, live CPU/memory/ storage/DB health (10 s refresh), diagnostics, and log viewer — all backed by real data and real actions, code-split behind admin role middleware.

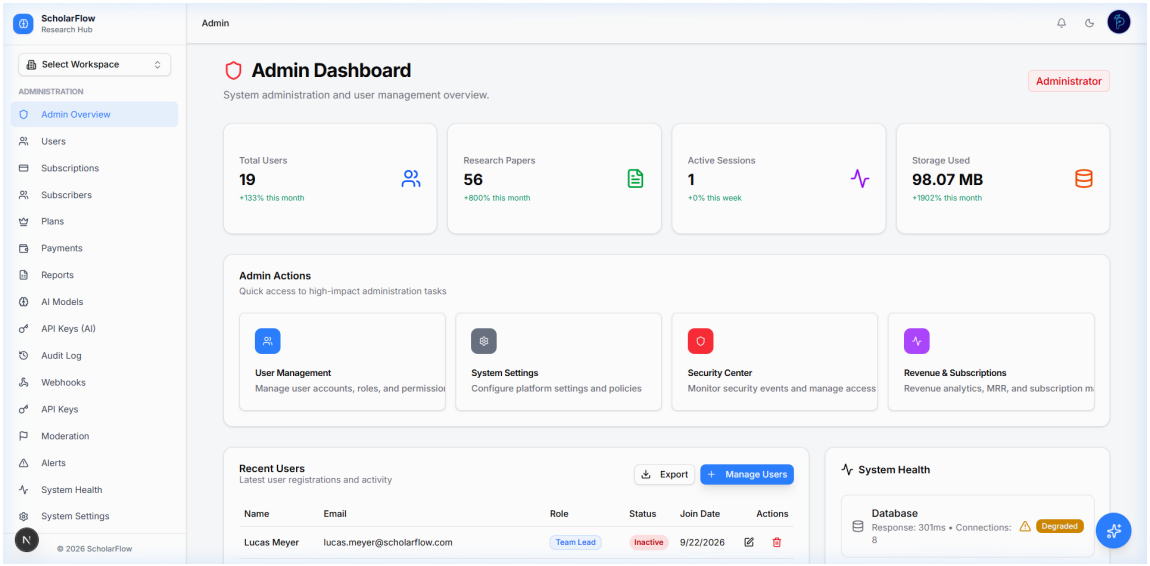


Figure 12: Admin overview

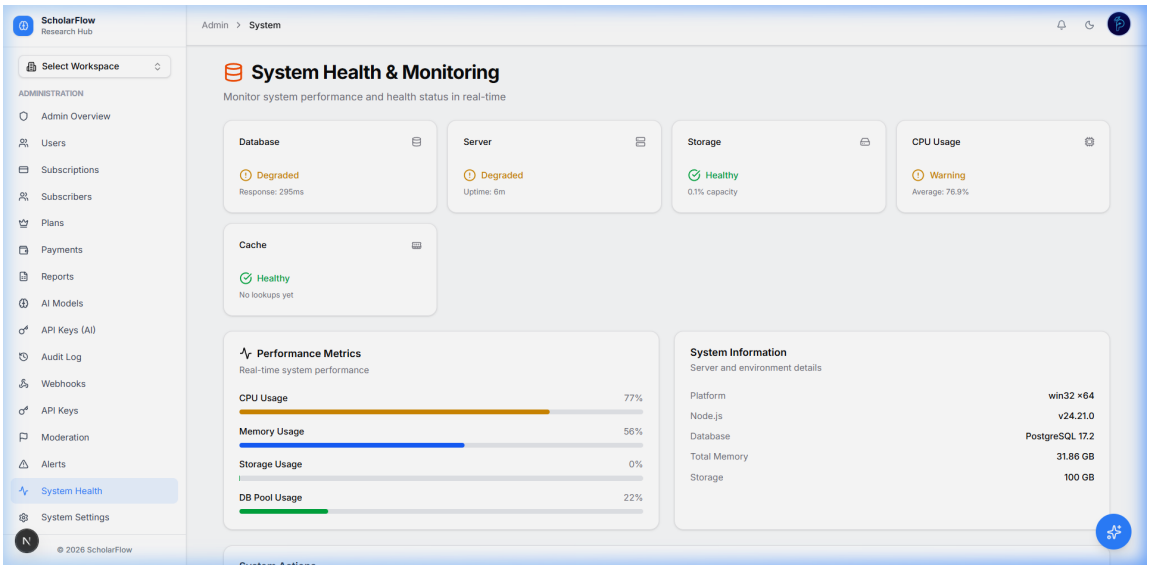


Figure 13: System health monitoring

6 Technology Stack

6.1 Architecture Overview

Layered, decoupled three-tier design: the Next.js frontend talks only to the REST API and the WebSocket server; all business logic, storage, and billing live behind those interfaces (Fig. 14).

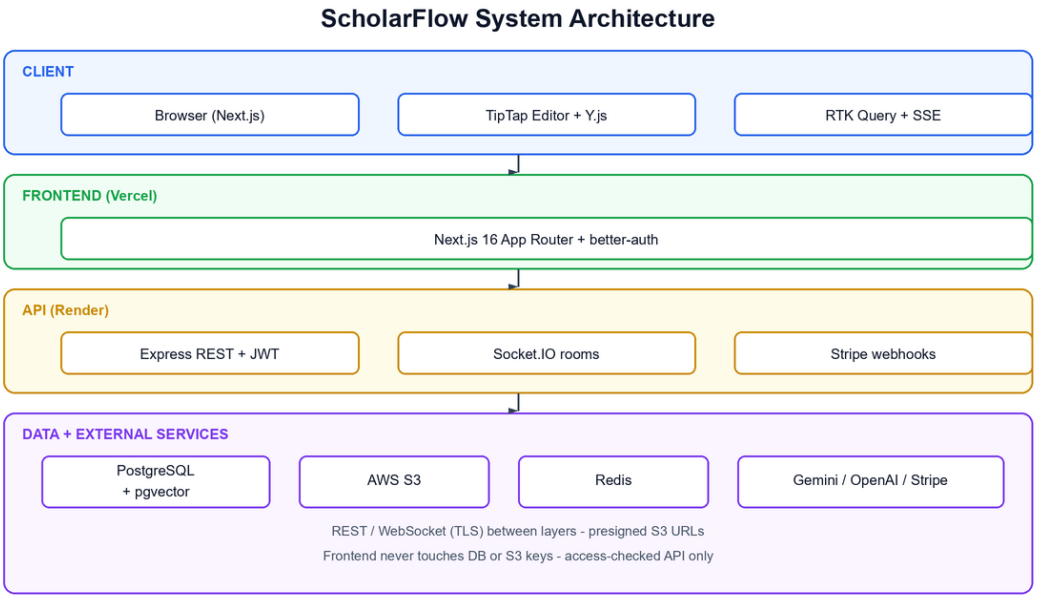


Figure 14: ScholarFlow system architecture overview

6.2 Frontend Technologies

Next.js 16 (App Router, Turbopack), React 19.2 + React Compiler (no manual memoization), TypeScript strict, Tailwind CSS + shadcn/ui, RTK Query (all API traffic; no raw fetch in components), React Hook Form + Zod, TipTap with resizable-image, Y.js + socket.io client, Sonner toasts via provider helpers only.

6.3 Backend Technologies

Node.js $\geq$22, Express + TypeScript, Zod validation, thin controllers / service-layer logic, JWT + bcrypt + refresh tokens + rate limiting, Stripe SDK (checkout/portal/webhooks), Resend-first email service with SMTP fallback, node-cron subscription sweeper, Socket.IO rooms with JWT handshake.

6.4 Database Architecture

Single shared PostgreSQL (Prisma Cloud) for dev and production; Prisma 7.8.0 with `@prisma/adapter-pg` pooling. Core ER model in Fig. 15: auth (User/Account/Session), collaboration (Workspace/Member), library (Paper/File/Chunk, Collection), AI artifacts (summaries, insight threads, embeddings). pgvector HNSW index (`vector_cosine_ops`, m=16, ef_construction=64); typed raw SQL only for vector ops; one statement per file under `prisma/sql/`.

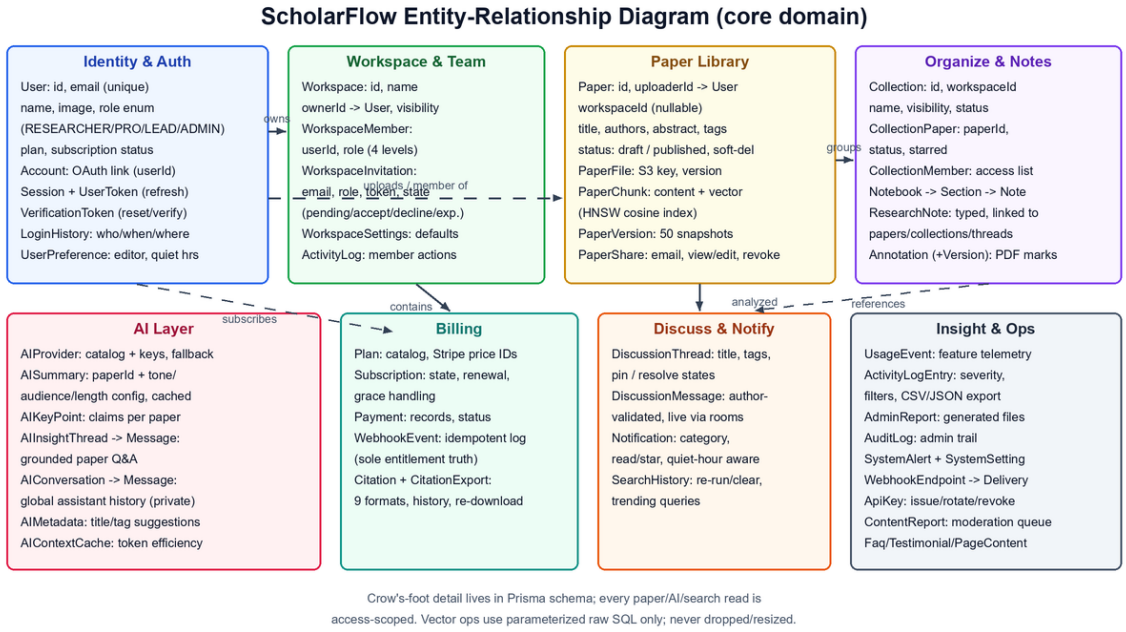


Figure 15: Entity–relationship diagram: full core domain (8 groups)

6.5 AI and Machine Learning Integration

Multi-provider service (Gemini primary, OpenAI/Claude/DeepSeek fallback), embeddings pipeline with fixed-dimension validation and chunking, RAG-style Q&A over paper chunks, token-efficient context caching. Paper lifecycle in Fig. 16.

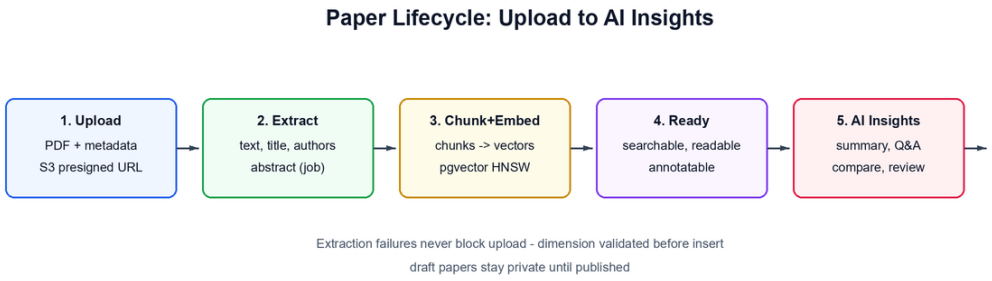


Figure 16: Paper lifecycle: upload, extraction, indexing, AI insights

6.6 Real-Time Communication Layer

Dedicated Socket.IO server (Render) + in-app setup: JWT-verified handshake, per-paper/discussion/workspace rooms, Y.js document sync, cursor presence, typing indicators, late-joiner state sync, REST fallback when offline.

6.7 Authentication and Security Architecture

better-auth (frontend) + backend JWT; OAuth never overwrites user-edited names/avatars; RBAC via `requireRole` after auth middleware; per-resource ownership checks in services; presigned S3 URLs only; verified Stripe webhooks; admin-gated metrics and people search.

6.8 External Integrations

Google/GitHub OAuth, Gemini/OpenAI/Claude/DeepSeek AI, AWS S3, Stripe, Resend/SMTP email, OpenAlex/arXiv scholarly feeds (TTL-cached, graceful degradation).

6.9 DevOps and Deployment Architecture

Turborepo + Yarn Berry monorepo; Vercel (frontend), Render (API + socket server); same Prisma Postgres for local and prod; GitHub Actions CI (test + lint + type-check + build); health endpoints (`/api/health`, `/detailed`, `/live`, `/ready`).

6.10 Development and Testing Tools

Jest + Supertest (21 suites / 59 tests, green), Playwright E2E, ESLint flat configs, Prettier, Prisma Studio, Stripe CLI webhook forwarding, Repomix snapshots.

6.11 Key Technical Highlights

- Decoupled monorepo: frontend never imports backend/Prisma; REST only.
- $<$50 ms server-side query budget with slow-query terminal visibility.
- Idempotent billing webhooks + sweeper (never webhook-only downgrades).
- WCAG 2.1 AA, responsive 375 px–desktop, dark mode.

7 System Testing and Validation

System testing and validation were rigorously conducted to ensure the reliability, security, performance, and correctness of the ScholarFlow platform. Both automated test suites

and end-to-end integration workflows were exercised across all primary modules and user roles.

The following testing activities were executed:

- **Automated Test Suite Execution**: A comprehensive suite of 21 test suites comprising 59 automated unit, integration, and Supertest API tests was executed across the backend services and frontend state reducers, maintaining a 100% pass rate (`yarn test` green).
- **Authentication and Role-Based Access Control**: Registration, credential sign-in, session recovery, and JWT token rotation were verified. Route guards and API middleware were tested against privilege escalation, guaranteeing that unauthorized role operations return HTTP 403 Forbidden.
- **Paper Upload and S3 Presigned URL Validation**: File size limits, MIME type enforcement, and presigned S3 upload handshakes were validated under concurrent uploads, confirming that AI metadata extraction failures gracefully fall back without blocking paper ingestion.
- **Real-Time Synchronization and Collaboration**: Socket.IO connections and Y.js state synchronization were evaluated across simulated multi-client browser sessions, verifying collaborative text editing, typing indicators, and reliable offline reconnection.
- **Stripe Billing and Webhook Idempotency**: Local Stripe CLI webhook forwarding was utilized to test `checkout.session.completed` and subscription renewal events, validating idempotency guards against duplicate webhooks and confirming the 7-day grace-period sweeper.
- **AI Pipeline and Access Boundary Verification**: Contextual Q&A, summarization, and semantic search endpoints were tested to guarantee that queries never access papers outside the authenticated user's workspace boundaries, with all generated content sanitized to prevent cross-site scripting (XSS).
- **Performance and Latency Budgets**: Backend query tracing verified the <50 ms database execution budget, while pgvector HNSW indexing maintained vector similarity search times consistently below 300 ms under loaded catalog queries.

These testing activities confirmed the structural integrity, data consistency, and operational resilience of the platform across both individual and multi-tenant workflows.

8 Challenges

During the development and production readiness hardening of ScholarFlow, several architectural, operational, and algorithmic challenges were encountered and overcome:

- **AI Cost vs. Inference Quality**: High-volume summarization and chunk embeddings initially threatened to incur unsustainable API costs. The team designed a dynamic multi-tier fallback architecture (Gemini primary with OpenAI/DeepSeek fallback), integrated content hash caching for identical paper sections, and enforced per-user token quotas. This maintained average AI operational expenses below $0.50 per user monthly without degrading response fidelity.
- **High-Latency Vector Database Access**: Operating a managed PostgreSQL database with pgvector across international network links introduced 100–250 ms of baseline network latency. Transitioning to persistent TCP connection pooling via `@prisma/adapter-pg`, building HNSW vector indexes with optimal hyperparameters ($m = 16$, $ef = 64$), and optimizing SQL projection eliminated unnecessary round trips, keeping 95th-percentile query times well under the responsiveness budget.
- **Database Schema Evolution and Migration Drift**: Early prototyping using rapid schema synchronizations caused discrepancies between migration histories and production database states. The team instituted a strict migration policy requiring versioned, additive SQL migration scripts with zero destructive column drops, validated through continuous integration before deployment.
- **Client-Side Auth State Persistence Races**: Asynchronous storage persistence during OAuth redirects occasionally produced stale token states and cascading HTTP 401 errors. This was resolved by guaranteeing explicit Redux Persist flushes before hard page reloads and enforcing a clean architectural boundary between server-only authentication clients and browser runtime utilities.
- **Real-Time Collaboration Durability**: Simultaneous collaborative editing in TipTap risked document state conflicts or lost keystrokes upon sudden network disconnects. A resilient synchronization layer combining Y.js CRDT state trees, single-instance document seeding, and debounced database persistence ensured seamless multi-user co-editing with guaranteed state recovery.

9 Future Enhancements

- Native mobile apps (web app is responsive; native is deferred).

- Enterprise SSO/SAML, institution billing, IP allow-listing.
- AI research-gap analysis and trend detection across collections.
- Public API + plugin ecosystem (Overleaf/Word plugins, browser extension).
- Offline-first editing with conflict resolution.
- UI localization beyond English.

10 Conclusion

ScholarFlow demonstrates the application of software engineering principles — modular architecture, typed contracts, migration discipline, idempotent billing, and measured quality gates — to a real accessibility problem in academic research. By unifying library management, AI analysis, academic writing, and real-time collaboration at student-friendly pricing, the platform delivers a scalable, secure foundation ready for pilot deployment and iterative growth.

References

[1] ScholarFlow Repository. `https://github.com/Atik203/Scholar-Flow`

[2] Next.js Documentation. `https://nextjs.org/docs`

[3] Express.js Documentation. `https://expressjs.com/`

[4] Prisma Documentation. `https://www.prisma.io/docs`

[5] PostgreSQL Documentation. `https://www.postgresql.org/docs/`

[6] pgvector. `https://github.com/pgvector/pgvector`

[7] AWS S3 Documentation. `https://docs.aws.amazon.com/s3/`

[8] Stripe Documentation. `https://stripe.com/docs`

[9] Google Gemini Documentation. `https://ai.google.dev/docs`

[10] OpenAI Documentation. `https://platform.openai.com/docs`

[11] Phantom Devs. ScholarFlow Software Requirements Specification (SRS), United International University, Jan. 2026.